

Bootcamp Databricks

Engenharia de Dados dentro da Plataforma Data & AI

I&D Capgemini Brasil, Setembro/2026

Agenda

Aulas	Assunto
Dia 1 – 14/09/2026	Fundamentos de Engenharia de dados e Databricks
Dia 2 – 15/09/2026	SQL, bancos, tabelas e Delta Lake
Dia 3 – 16/09/2026	Unity Catalog, schemas, volumes e governança
Dia 4 – 17/09/2026	Ingestão de dados: Batch, Incremental, Auto Loader
Dia 5 – 18/09/2026	ETL vs ELT & transformação arquitetura medalhão
Dia 6 – 21/09/2026	Modelagem de dados para Analytics
Dia 7 – 22/09/2026	Processamento distribuído com Spark no Databricks
Dia 8 – 23/09/2026	Streaming e processamento contínuo
Dia 9 – 24/09/2026	Orquestração, Jobs, DataOps e boas práticas
Dia 10 – 25/09/2026	Qualidade, observabilidade, arquitetura e desafio final

Dia 8 - Streaming e processamento contínuo

Streaming e processamento contínuo

I&D Capgemini Brasil, Setembro/2026



DATBRICKS HANDS-ON LAB

STRUCTURED STREAMING

readStream de arquivos → parse JSON → writeStream para Delta

Checkpoint, trigger por intervalo e monitoramento em tempo real



NEAR REAL-TIME

Dados processados em micro-batches



EXACTLY-ONCE

Garantia de processamento e entrega exatamente-uma-vez



ESCALÁVEL

Processamento distribuído e elástico



OBSERVABILIDADE

Spark UI e métricas em tempo real

FONTE DE DADOS

Novos arquivos JSON chegando na pasta de entrada



READ STREAM

readStream from files



PARSE JSON

Schema e extração de campos



TRANSFORMAÇÕES

Filtros, seleções, enriquecimentos



WRITE STREAM

writeStream to Delta table



DELTA TABLE

Tabela Delta de destino



CONSUMO / BI

Dashboards e consultas em tempo real



NOTEBOOK: structured_streaming_lab

```
# 1. Imports e configurações
from pyspark.sql import functions as F
from pyspark.sql.types import *

# 2. Definição do schema do JSON
schema = StructType([
    StructField("event_id", StringType()),
    StructField("user_id", StringType()),
    StructField("event_type", StringType()),
    StructField("event_ts", TimestampType()),
    StructField("country", StringType()),
    StructField("value", DoubleType())
])

# 3. Leitura stream dos arquivos JSON
df_raw = (spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "json")
    .load("/mnt/raw/events/"))

# 4. Parse JSON com schema
df_parsed = (df_raw.select(F.from_json(F.col("value"), schema).alias("data"))
    .select("data.*"))

# 5. Transformações
df_transformed = (df_parsed
    .withColumn("event_date", F.to_date("event_ts"))
    .filter(F.col("event_type").isin("click", "purchase"))
    .withColumn("ingestion_ts", F.current_timestamp()))

# 6. Escrita stream para Delta com checkpoint e trigger
query = (df_transformed.writeStream
    .format("delta")
    .outputMode("append")
    .option("checkpointLocation", "/mnt/checkpoints/events_stream/")
    .trigger(processingTime="10 seconds")
    .toTable("analytics.events_stream"))

# 7. Iniciar stream
query.awaitTermination()
```

CHECKPOINT (ESTADO)

Garante fault tolerance e exactly-once



TRIGGER

Processamento contínuo



processingTime = "10 seconds"

Micro-batch a cada 10s

ARQUITETURA DE EXECUÇÃO

DRIVER



SparkSession
Query Plan
Scheduler
Checkpoint
Catalog

EXECUTORS (Cluster)



Cache / Memory



Cache / Memory



Cache / Memory



Cache / Memory



ACID

Transações confiáveis e consistentes



TIME TRAVEL

Histórico e rollback de dados



SCHEMA EVOLUTION

Evolução de schema automática



OPTIMIZED

Compactação e desempenho

MONITORAMENTO - SPARK UI STREAMING

Overview

Jobs

Stages

SQL

Streaming

Streaming Query

RUNNING

uptime: 2h 35m 18s

Input Rate
10.2 rows/s



Processed Rate
10.1 rows/s



Batch Duration
1.3 s



State Operators
2



Recent Batches

Batch Id	Input Rows	Processed Rows	Duration	Status
1257	101	101	1.2 s	✓
1256	98	98	1.3 s	✓
1255	105	105	1.4 s	✓
1254	99	99	1.3 s	✓
1253	97	97	1.2 s	✓

State Store



Operator Breakdown



databricks



PYSPARK

APIs familiares e poderosas



DELTA LAKE

Armazenamento aberto e confiável



STREAMING

Processamento contínuo e escalável



LAKEHOUSE

Unificando data lakes e data warehouses



DICA

Ajuste o trigger conforme a necessidade de negócio. Menor intervalo = menor latência, maior custo.



Objetivos do Dia 8

Sair do batch e entender quando near real-time realmente faz sentido.

- Diferenciar batch, incremental e streaming contínuo.
- Explicar o modelo mental de Structured Streaming no Databricks.
- Entender checkpoint, trigger, output mode, janelas e late events.
- Construir uma leitura stream e escrita em Delta.
- Simular ingestão contínua e entregar um pipeline streaming simples.

Mensagem central

Streaming não é batch mais rápido. É um contrato operacional com estado, latência e custo.

Entregável

Pipeline streaming simples com checkpoint, trigger e tabela Delta de saída.

Recap: de batch distribuído para streaming

O Day 7 explicou o motor; o Day 8 muda o modo de chegada dos dados.

Day 7	Day 8	Mudança real	Foco
Batch processing com PySpark e Spark distribuído.	Eventos chegando continuamente e resultados atualizados.	Estado, checkpoint, trigger, latência e late events.	Near real-time.

- No batch, você processa um conjunto delimitado.
- No streaming, você processa uma fonte que continua recebendo novos dados.
- O mesmo Spark executa, mas a operação exige novas decisões de desenho.

Por que streaming?

Streaming só se justifica quando a latência do dado muda a decisão de negócio.

Operação

Monitorar eventos e agir rapidamente.

Experiência

Atualizar alertas, dashboards e aplicações em menor latência.

Risco

Detectar padrões e anomalias antes do próximo lote.

- Use streaming quando esperar o próximo lote compromete o valor do dado.
- Não use streaming apenas porque parece moderno.
- Streaming adiciona estado, custo operacional e necessidade de monitoramento contínuo.

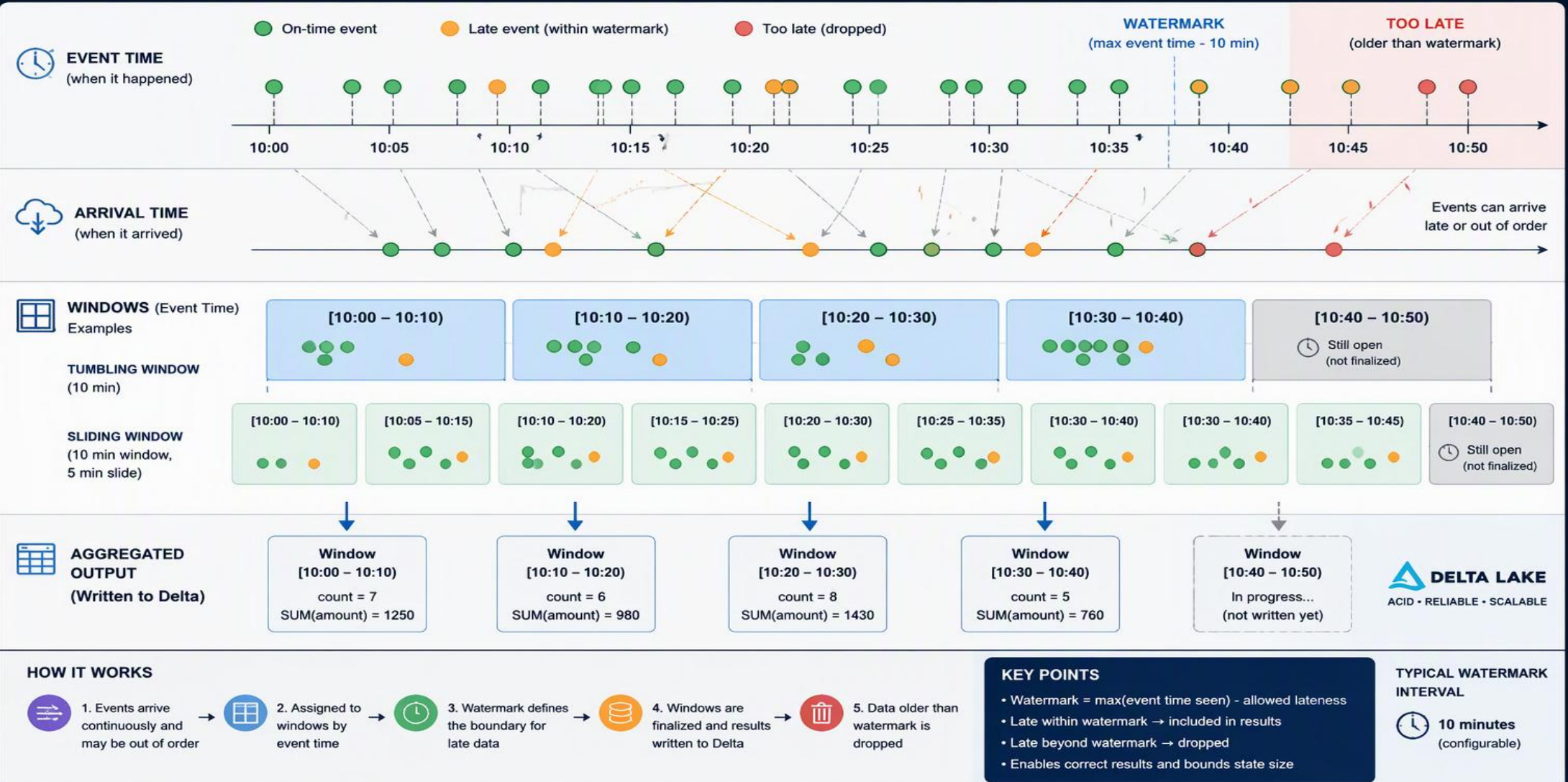
Streaming vs Batch

Streaming vs Batch



EVENT TIME WINDOWS & LATE EVENTS

From Event Time to Final Results in Databricks Lakehouse



Batch, incremental e streaming

Antes de codificar, escolha o contrato correto de chegada e processamento.

Critério	Batch	Incremental	Streaming
Chegada	Lotes fechados	Novos dados por janela/execução	Eventos contínuos
Execução	Job sob demanda ou agenda	Job recorrente com estado leve	Query contínua ou micro-batch
Latência	Minutos, horas ou dias	Minutos ou horas	Segundos ou minutos
Estado	Geralmente simples	Controle de processado	Checkpoint, offsets e estado
Uso típico	Relatórios e cargas históricas	Arquivos novos, CDC, backfills	Alertas, monitoramento, IoT, near real-time

Event-driven: pensar em eventos

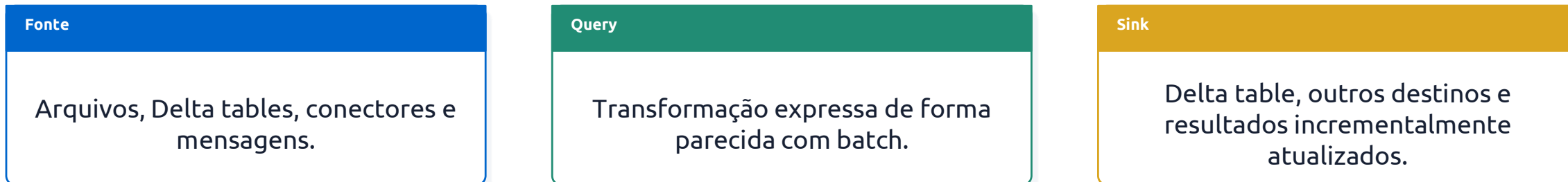
Evento é algo que aconteceu e que pode alterar uma visão analítica ou operacional.

Evento	Payload	Contrato
Uma compra, clique, leitura de sensor, transação ou log.	Dados do evento: id, timestamp, tipo, atributos e valor.	Formato, schema, retenção, chave e semântica de tempo.

- Streaming começa na qualidade do evento, não na ferramenta.
- Timestamp de evento e timestamp de chegada podem ser diferentes.
- Se o contrato do evento é ruim, a pipeline streaming vira correção permanente de sujeira.

Structured Streaming no Databricks

Uma stream pode ser tratada como uma tabela que recebe novas linhas continuamente.



- Structured Streaming permite expressar computação em dados streaming de forma semelhante a batch.
- O motor executa incrementalmente e atualiza resultados conforme dados chegam.
- A base conceitual é simples; a operação exige disciplina.

Modelo readStream → writeStream

Streaming é uma query contínua: fonte, transformação, destino e estado.



1: Create streaming events table

```
1 stream_df = (spark.readStream
2   .format('cloudFiles')
3   .option('cloudFiles.format', 'json')
4   .option('cloudFiles.schemaLocation', '/Volumes/capgemini_academy/landing/metadata/_schema/events')
5   .load('/Volumes/capgemini_academy/landing/files/events'))
6
7 query = (stream_df.writeStream
8   .format('delta')
9   .option('checkpointLocation', '/Volumes/capgemini_academy/landing/metadata/_checkpoint/events')
10  .trigger(availableNow=True)
11  .toTable('capgemini_academy.bronze.events_stream'))
```

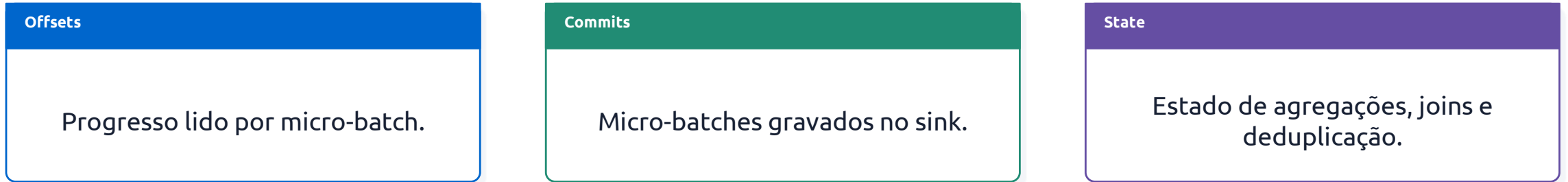

Fontes e destinos em streaming

A arquitetura precisa definir de onde os eventos vêm e onde o resultado fica.

Elemento	Exemplos	Decisão Importante
Fonte	Arquivos chegando em volume ou storage	Schema, frequência, retenção
Transformação	Parse, filtro, agregação, enriquecimento	Stateless ou stateful
Destino	Tabela Delta Bronze/Silver/Gold	Output mode e idempotência
Estado	Checkpoint e state store	Local único por query
Observabilidade	Spark UI e métricas	Latência, backlog e falhas

Checkpoint: memória operacional da stream

Checkpoint não é pasta temporária. É o estado que permite recuperar a query.



- Cada query deve ter checkpoint próprio.
- Trocar ou apagar checkpoint faz a próxima execução começar como nova query.
- Mudanças em fonte, sink ou estado podem exigir novo checkpoint.
- Checkpoint deve viver em local governado, estável e controlado.

Trigger: frequência de processamento

Trigger controla quando a query verifica e processa novos dados.

Trigger	Uso didático	Observação
Default	Streaming geral	Processa conforme dados chegam
Processing time	Micro-batches em intervalo fixo	Balanceia latência e custo
AvailableNow	Incremental batch	Processa dados disponíveis e encerra
Real-time mode	Muito baixa latência	Use apenas se o caso exigir

```
query = (df.writeStream
    .trigger(processingTime='10 seconds')
    .option('checkpointLocation', checkpoint_path)
    .toTable(target_table))
```

Output modes

O output mode define o que será gravado no destino a cada atualização.

Append

Grava apenas novas linhas finalizadas.

Update

Grava linhas alteradas desde o último trigger.

Complete

Grava o resultado completo agregado.

- Append é comum para ingestão de eventos sem agregação mutável.
- Update aparece em cenários com agregação incremental.
- Complete pode ser caro para agregações grandes.
- Escolha output mode considerando estado, sink e semântica do resultado.

Stateless vs Stateful

A diferença é se a query precisa lembrar do passado para calcular o presente.

Stateless

Processa cada linha sem manter estado intermediário.
Exemplos: select, filter, parse e enriquecimento simples.

Stateful

Mantém estado para agregações, joins, deduplicação e janelas. Exige checkpoint e gestão de estado.

- Stateful é poderoso, mas aumenta custo e complexidade operacional.
- Watermark ajuda a controlar quanto tempo manter estado em operações com eventos atrasados.
- Em laboratório, identifique claramente se a transformação é stateless ou stateful.

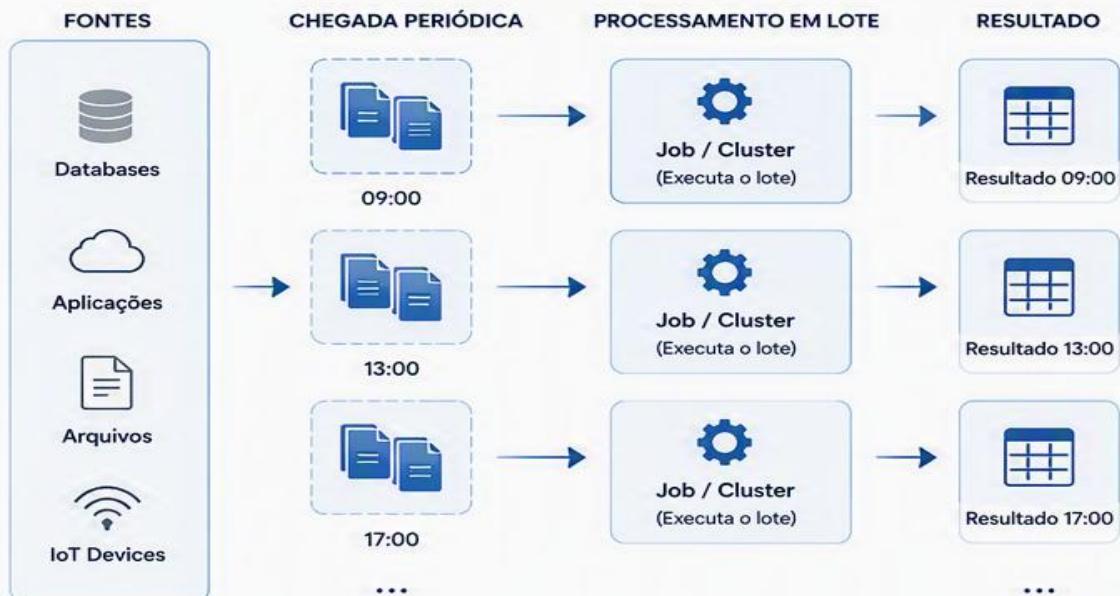
STREAMING vs BATCH

Duas abordagens para processar dados em escala no Databricks



BATCH PROCESSING

Dados são coletados em intervalos e processados em lotes



IDEAL PARA

Grandes volumes, ETL pesado, relatórios e análises históricas



LATÊNCIA

Minutos a horas (depende do intervalo do lote)



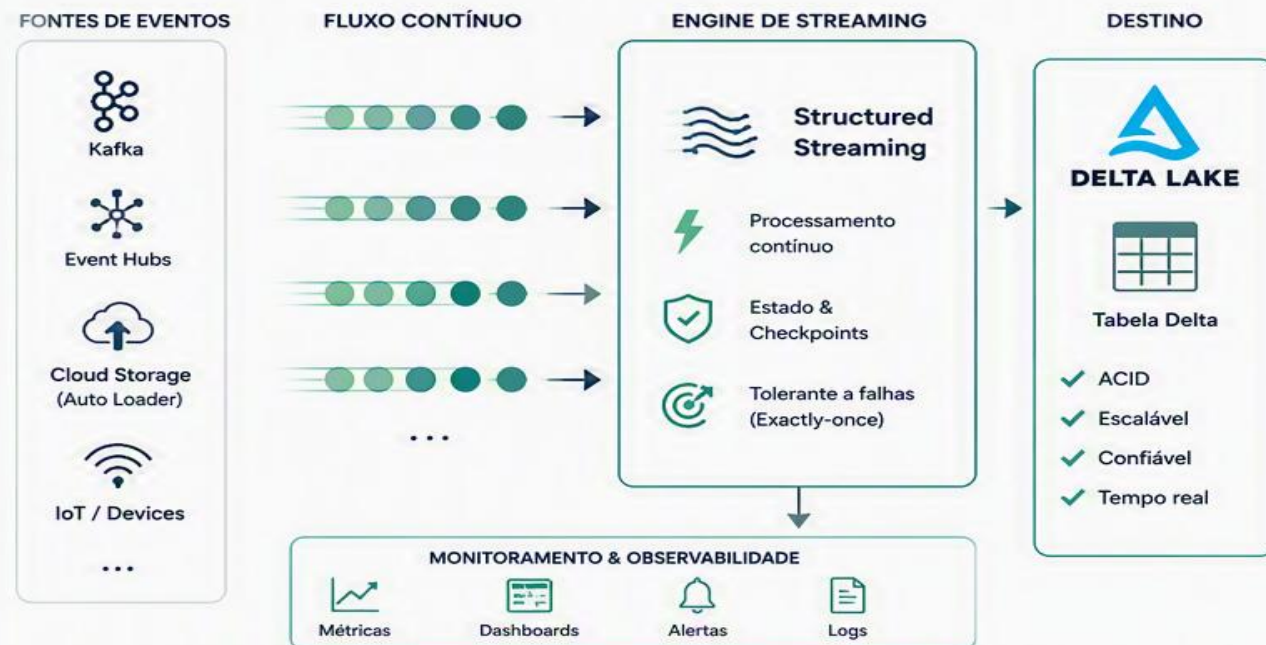
EXEMPLOS

ETL diário, carga de dados, reprocessos, data warehouse



STREAMING PROCESSING

Eventos são processados continuamente em tempo real (ou quase tempo real)



IDEAL PARA

Dados em tempo real, dashboards ao vivo, detecção de fraude, monitoramento, IoT



LATÊNCIA

Milissegundos a segundos (quase tempo real)



EXEMPLOS

Streaming de cliques, telemetria, sensores, métricas operacionais

AMBOS CONVERGEM NO DELTA LAKE



Fontes



Processamento
(Batch ou Streaming)



Delta Lake



Camada Gold
(Consumo Analítico)



BI / Analytics
/ ML



ESCOLHA A ABORDAGEM CERTA

Batch para eficiência em grandes volumes.
Streaming para agilidade e tempo real.
Muitas soluções usam ambos de forma complementar.

Exemplo: agregação por janela

O resultado muda conforme novos eventos chegam dentro da janela permitida.

```
from pyspark.sql.functions import window, col, count

events = spark.read.table(bronze_table)

agg = (events
      .withWatermark('event_ts', '10 minutes')
      .groupBy(window('event_ts', '5 minutes'), col('event_type'))
      .agg(count('*').alias('events_count'))
      )

agg.display()
events.display()
```

- window define o intervalo de agregação.
- withWatermark define quanto atraso será aceito para dados tardios.
- Sem watermark, o estado de uma agregação temporal pode crescer indefinidamente.
- Late events são decisão de negócio e tecnologia, não detalhe de código.

Late events: o dado chegou depois

O evento pode ter acontecido dentro da janela, mas chegou tarde ao pipeline.

Event time	Arrival time	Watermark
Quando o evento aconteceu.	Quando o evento chegou ao sistema.	Limite usado para aceitar ou descartar atraso.

- Não confunda horário de chegada com horário do evento.
- Uma venda feita às 10:00 pode chegar às 10:07 por falha de rede ou atraso de origem.
- A regra de atraso aceitável precisa vir do negócio e da operação.

Lambda e Kappa: visão conceitual

Arquiteturas para combinar histórico, eventos e atualização contínua.

Lambda

Combina camada batch para histórico e camada streaming para baixa latência. Pode aumentar complexidade por manter dois caminhos.

Kappa

Tenta simplificar usando um caminho orientado a eventos/streaming para histórico e fluxo. Exige forte disciplina de retenção e replay.

- Não transforme Lambda/Kappa em dogma. Use como referência de desenho.
- No Lakehouse, Delta e Structured Streaming permitem composições híbridas.
- A pergunta prática é:
preciso reprocessar histórico, operar near real-time ou ambos?

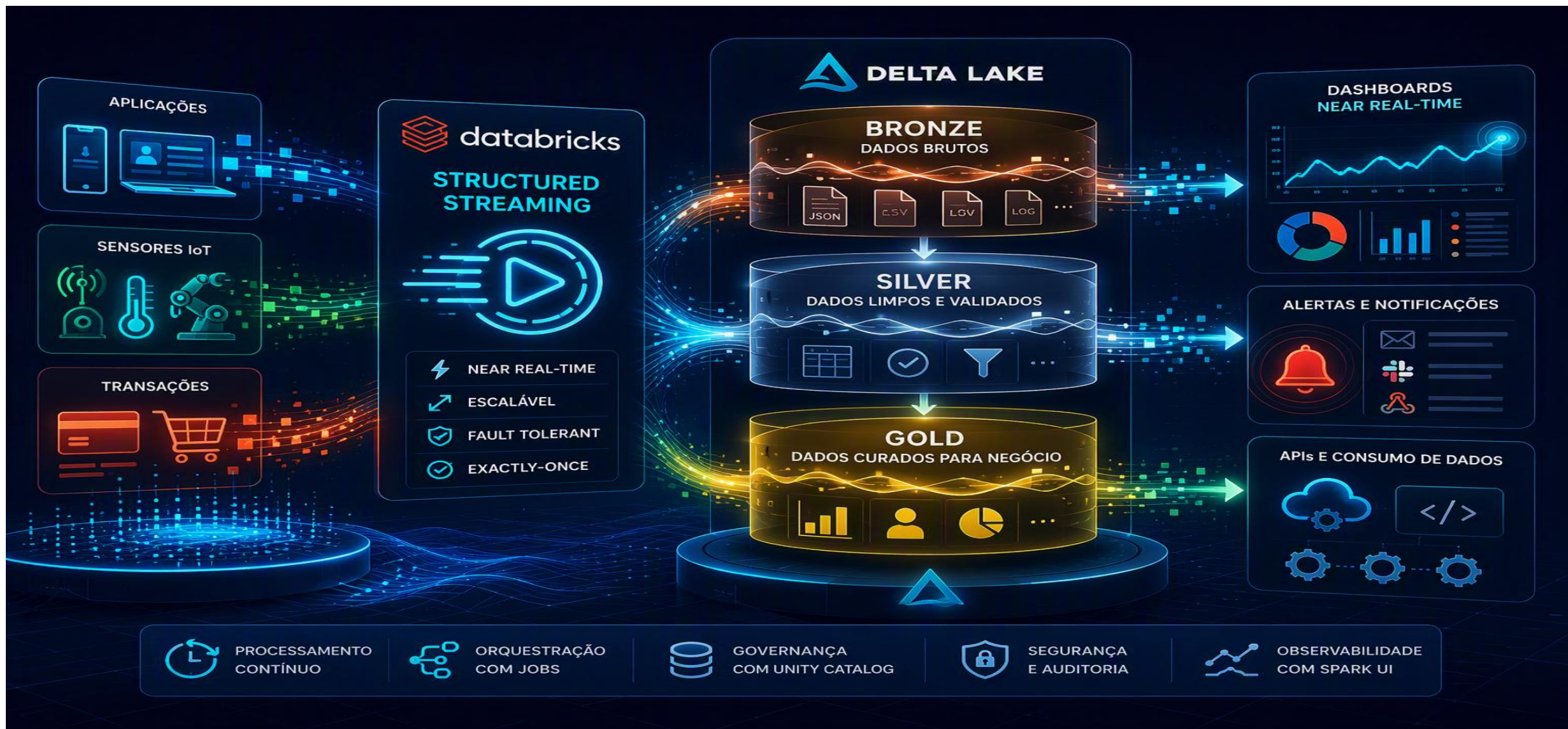
Quando vale a pena usar streaming?

A decisão deve passar por valor, latência, custo e operação.

Pergunta	Se a resposta for sim	Possível decisão
A decisão perde valor se esperar o lote?	Sim	Considere streaming
A fonte envia eventos continuamente?	Sim	Considere stream ou incremental
Existe equipe para operar estado e falhas?	Não	Cuidado com streaming
A latência de minutos é suficiente?	Sim	Micro-batch pode bastar
É só carga diária?	Sim	Batch ou AvailableNow pode ser melhor

Arquitetura do laboratório

Simular ingestão contínua, processar eventos e gravar resultados em Delta.



Objetivo: Simular pedidos chegando continuamente em arquivos JSON, processar o envelope events com Structured Streaming, gravar eventos em Delta e produzir uma agregação por janela com checkpoint, watermark e validação.

Narrativa: no Dia 7 entendemos como o Spark executa planos. No Dia 8 o mesmo motor passa a processar uma fonte que continua recebendo novos eventos.

Jornada prática do laboratório

1 Setup governado

SET

Definir source, checkpoint e tabelas no catálogo capgemini_academy.

2 Chegada de eventos

JSON

Copiar arquivos JSON em lotes para simular pedidos contínuos.

3 readStream

READ

Ler o envelope JSON com schema explícito para o array events.

4 Explode + tipagem

PARSE

Explodir events, tipar event_ts e adicionar ingestion_ts.

5 Bronze Delta

BRZ

Gravar novos eventos em bronze.events_stream usando append.

6 Watermark + janela

TIME

Agrupar purchases em janelas de 5 min aceitando atraso de 10 min.

7 Gold near real-time

GOLD

Gravar receita e quantidade por janela em tabela Delta agregada.

8 Observabilidade

OBS

Validar progresso, checkpoint, contagens e chegada tardia.

Estrutura esperada

capgemini_academy

LANDING

events_001...010
JSON batches



BRONZE

events_stream
raw events



GOLD

purchase_revenue
_5min

readStream → writeStream

Checklist de validação

- ✓ 10 arquivos JSON organizados em lotes
- ✓ Array events explodido corretamente
- ✓ event_ts convertido para timestamp
- ✓ Cada query com checkpoint exclusivo
- ✓ Bronze cresce sem duplicar arquivos
- ✓ Agregação de 5 min validada

Comandos centrais

```
raw = spark.readStream.schema(envelope_schema).json(source)
events = raw.select(F.explode("events").alias("e"))
events = events.select("e.*").withColumn("event_ts", ...)
events.writeStream.outputMode("append").toTable(bronze)
events.withWatermark("event_ts", "10 minutes")
  .groupBy(F.window("event_ts", "5 minutes")).agg(...)
```

Streaming é decisão de arquitetura, não apenas outro comando.

Lab setup: parâmetros e paths

Separe fonte, checkpoint e destino. Nunca trate checkpoint como pasta descartável.

```
catalog = 'capgemini_academy'
schema_volume = 'landing'
schema_bronze = 'bronze'

source_path = f'/Volumes/{catalog}/{schema_volume}/files/events'
checkpoint_path = f'/Volumes/{catalog}/{schema_volume}/metadata/_checkpoint/events'

target_table = f'{catalog}.{schema_bronze}.events_stream'
```

- Crie paths explícitos para fonte, checkpoint e destino.
- Use nomes alinhados ao catálogo/schema de laboratório.
- O checkpoint deve ser único para a query.

Lab: simular ingestão contínua

Para treinamento, novos arquivos JSON podem simular eventos chegando.

```
# Exemplo didático: gravar pequenos arquivos JSON na pasta de entrada
```

```
{"event_id":"e1","event_ts":"2026-07-30T10: 00: 00","event_type":"click","value": 10.0}  
{"event_id":"e2","event_ts":"2026-07-30T10: 01: 00","event_type":"purchase","value": 99.0}
```

- O objetivo é observar a stream processando novidades incrementalmente.
- Use dados fictícios no laboratório.
- Mantenha o schema simples para focar em streaming, não na complexidade da fonte.

Lab: readStream de arquivos

Usar arquivos permite simular streaming sem depender de Kafka/Event Hubs no laboratório.

```
from pyspark.sql.types import *
from pyspark.sql.functions as F

schema_events = StructType([
    StructField('event_id', StringType()),
    StructField('pedido_id', StringType()),
    StructField('cliente_id', StringType()),
    StructField('produto_id', StringType()),
    StructField('event_type', StringType()),
    StructField('valor_total', DoubleType()),
    StructField('event_ts', StringType())
])

raw_stream = (spark.readStream
    .schema(schema_events)
    .json(source_path))

print(raw_stream )
```

Lab: transformação da stream

Parse, tipagem e colunas técnicas antes da escrita em Delta.

```
events_stream = (raw_stream
    .withColumn('event_ts', to_timestamp('event_ts'))
    .withColumn('event_date', to_date('event_ts'))
    .withColumn('ingestion_ts', current_timestamp())
    .filter(F.col('event_id').isNotNull())
)
```

- Tipagem de timestamp é essencial para janelas.
- Inclua ingestion_ts para rastreabilidade operacional.
- Filtros simples podem ser stateless.
- Agregações com janela serão stateful.

Lab: writeStream para Delta

A escrita materializa a stream e usa checkpoint para continuidade.

```
query = (events_stream.writeStream
  .format('delta')
  .outputMode('append')
  .option('checkpointLocation', checkpoint_path)
  .trigger(availableNow=True)
  .toTable(target_table))
```

- outputMode append grava novos eventos.
- checkpointLocation guarda progresso da query.
- trigger define o intervalo de micro-batch.
- toTable grava em tabela Delta governada.

Lab: resultado agregado por janela

Agora a query mantém estado para calcular contagens por janela de tempo.

```
agg_result = (events_stream
    .withWatermark('event_ts', '10 minutes')
    .groupBy(window('event_ts', '5 minutes'), 'event_type')
    .agg(
        count('*').alias('events_count'),
        sum('value').alias('total_value')
    )
)
```

- Essa etapa é stateful.
- A watermark limita o tratamento de eventos atrasados.
- A tabela agregada pode alimentar dashboard ou monitoramento.
- Valide se a janela escolhida faz sentido para o negócio.

Lab: escrever agregados em Delta

A tabela de resultado representa o consumo near real-time do laboratório.

```
agg_table = f'{catalog}.{schema_silver}.events_agg_5min'
agg_checkpoint = f'/Volumes/{catalog}/{schema_volume}/_checkpoint/events_agg_5min'

agg_query = (agg_result.writeStream
    .format('delta')
    .outputMode('append')
    .option('checkpointLocation', agg_checkpoint)
    .trigger(availableNow=True)
    .toTable(agg_table)
)
```

- Use checkpoint diferente para cada query.
- A escolha do output mode deve ser compatível com a agregação e watermark.
- A tabela pode ser consultada enquanto a stream continua processando.

Monitoramento da stream

Streaming precisa de observabilidade contínua: latência, backlog, falha e estado.

Progresso

Rows input, processed rows, batch duration.

Estado

Tamanho do state store e operadores stateful.

Alertas

Falhas, atraso, custo e queda de throughput.

- Monitore se a taxa de processamento acompanha a taxa de chegada.
- Observe batch duration e estado em agregações/window.
- Em produção, monitore logs, métricas, alertas e SLA de dados.

Erros comuns em streaming

O problema geralmente não é o código rodar; é rodar corretamente todos os dias.

Erro 1

Reusar checkpoint entre queries diferentes.

Erro 2

Mudar fonte/sink/estado sem entender compatibilidade do checkpoint.

Erro 3

Criar streaming sem caso real de latência.

- Checkpoint é contrato operacional. Trate como ativo do pipeline.
- Streaming exige monitoramento.
Sem observabilidade, falhas viram silêncio.
- Escolha trigger conforme latência e custo, não por impulso.

Fechamento do Day 8

Você sai com um pipeline streaming simples e critérios para decidir quando usar streaming.

Conceito	Prática	Operação
Streaming vs batch, event-driven, janelas e late events.	readStream, writeStream, checkpoint, trigger e Delta.	Monitoramento, estado e cuidado com custo/latência.

- Entregável: pipeline streaming simples com resultado agregado.
- O próximo passo natural é levar pipelines para um desenho mais operacional com automação, governança e observabilidade.
- Não finalize a aula sem reforçar: streaming é decisão de arquitetura, não apenas um comando diferente.

Fim do Day 8

Thank you.

Make it real.

Capgemini 